

Evidence before permission

Status: proposed for team review. Prepared 26 September 2026 from assay-handover-1600.zip and the team's stated product direction. This is the full product specification; event scope is a deliberate subset. No implementation or spending is authorized by this document.

The product in one paragraph

Assay helps an agent owner decide what an AI agent may do, whether a different configuration is worth using, and whether a proposed correction actually improves behavior. It observes complete tasks, evaluates outcomes against a defined standard, and turns sufficient evidence into a scoped recommendation. A human approves consequential changes. Jira triage is the first demonstration adapter, not the boundary of the product.

Problem and promise

Activity logs show what an agent did, but a busy dashboard does not establish whether it should act without review. A cheaper model name does not establish a cheaper successful workflow. A correction that fixes one example does not establish a safe improvement. Assay connects these decisions to inspectable evidence, uncertainty, configuration versions, and an approval record.

Promise: give the owner a defensible next action: approve a narrowly defined permission, retain review, test a candidate, reject a regression, or gather more evidence. Assay does not promise universal agent safety or a globally optimal configuration.

The three product questions

Q1 - May it act? AUTO, SUGGEST or QUIET for a specific action, population and agent version. Lead with this in the event demo.

Q2 - Can it run cheaper? CERTIFY, REJECT, INSUFFICIENT or REFUSE for a complete candidate configuration against a baseline, using task-level quality and cost evidence.

Q3 - Did the correction help? KEEP, DISCARD or UNPROVEN for a proposed version, with fixes and regressions visible. KEEP remains a recommendation until approved.

Reading guide

Pages 2-3 define users and screens. Pages 4-6 define functional and evidence requirements. Pages 7-8 define architecture, data and operating requirements. Pages 9-10 define validation, scope and unresolved decisions. The companion delivery plan assigns these requirements to gated stages.

Who uses Assay, and how

Primary users

Agent owner: defines the job, acceptable errors, allowed actions, candidate configurations and spending limits. Owns deployment approval. Example: your dad operating an existing customer-support chatbot.

Reviewer: labels selected outcomes, reviews suggested actions, corrects mistakes and checks proposed improvements. The reviewer needs enough domain knowledge to assess correctness; a model's confidence is not a replacement.

Builder / operator: connects the agent, captures events, implements safe action execution and maintains the deployment. In the event team, one person may hold multiple roles; production permissions must distinguish them.

Your dad's chatbot: the concrete journey

1. Connect the chatbot through a supported adapter. For the first build, import a documented task file; a generic endpoint connector is a later stage. A chatbot URL alone is not sufficient access.
2. Define success using representative conversations, expected outcomes and an explicit rubric. Examples include a correct answer grounded in an approved source, appropriate escalation, and no unsupported refund commitment.
3. Import prior runs for inspection. To compare a new agent version, execute fresh tasks against that version or run an authorized prospective experiment. Do not substitute a different model into an old agent trajectory and call it observed evidence.
4. Assay shows runs in Live Analysis and proposes an experiment in Decision Orchestrator. The owner approves any paid test budget before execution.
5. Inspect outcomes and uncertainties. Assay might recommend retaining human review for refund actions while permitting a narrowly defined informational response. Each permission applies only to its evaluated scope.
6. Approve a supported change, retain a baseline or holdback where appropriate, and monitor outcomes. Expire or revoke permission if the configuration or relevant operating conditions change.

User stories and boundaries

U1: As an owner, I can inspect why an action was suggested and approve it once. **U2:** As a reviewer, I can record a correction without silently changing production. **U3:** As an owner, I can compare versions and understand insufficient evidence. **U4:** As a judge, I can try a case and follow it to an evidence receipt and sandbox action.

The event experience uses public Jira examples and Assay's own action table. It must not modify Apache Jira. Production write connectors, customer onboarding, billing and multi-tenant service operation are later work.

Observe work. Make decisions.

Dashboard 1 - Live Analysis

Purpose: answer what happened and what needs attention. Show the selected project, agent version, data mode, run status and last refresh. Display task volume, errors, latency, recorded usage and actual action counts. Every summary links to the underlying task records.

Task detail: input reference, retrieved evidence, proposed action, rationale, model/configuration, timestamps, usage provenance, reviewer outcome and action status. Sensitive content must be masked or omitted according to the project's data policy.

Review queue: pending cases with Approve, Reject and Correct. A rejection stores a reason. A correction creates evidence for a candidate change; it does not edit the active prompt or issue a new AUTO permission.

Live semantics: historical imports say RECORDED; synthetic examples say SYNTHETIC; fresh execution says LIVE. Playback is explicitly labeled. A failed or stale feed cannot display a live-success state.

Dashboard 2 - Decision Orchestrator

Purpose: answer what should change and why. Show active permissions, pending experiments, baseline/candidate versions, evaluated population, quality and cost evidence, uncertainty, verdict and required approval.

Decision detail: target action, target quality, sample and effective independence limits, evidence links, test/checkpoint version, exclusions, cost basis and reasons to refuse. Show the next useful action when evidence is insufficient; do not invent an exact task count or time estimate.

Change controls: approve eligible recommendation, decline with reason, pause a permission and roll back an active version. Controls must enforce state transitions in the backend, not merely hide buttons.

Shared navigation and minimal event implementation

Proposed navigation: Overview, Live Analysis, Decision Orchestrator, Evidence and Settings. Stage 1 may use two top-level tabs within the existing Streamlit app, with detail panels under each. The product has two distinct views; it does not need two services or two separately hosted websites.

Empty state: explain how to load the supplied demo. **Error state:** show failed operation and retry path without claiming success. **Insufficient evidence:** explain missing labels, sample support or invalid scope. **Conflict state:** tell the reviewer an action was already handled.

A decision must be traceable from summary to evidence and from evidence to execution. Counts of predictions eligible for AUTO must never be labeled as completed automatic actions.

Requirements / evidence and actions

Priority: P0 = event essentials; P1 = following increment; P2 = later product. Requirement IDs remain stable across both documents.

F01 - Project and version identity [P0]

Each task records project, agent, task ID, configuration ID, model settings and prompt version. Re-importing the same run is idempotent. An incomplete version is visible and cannot support a new permission. Acceptance: a changed prompt produces a different version and does not reuse cached judgments from the old version.

F02 - Evidence ingestion and provenance [P0]

Load the handover dataset through a documented schema. Validate required fields and separate rejected records with reasons. Preserve RECORDED / SYNTHETIC / LIVE and OBSERVED / ESTIMATED cost provenance. Acceptance: malformed records cannot silently enter certification; valid repeat imports do not duplicate counts.

F03 - Review and feedback [P0]

Store reviewer decision, reason, timestamp, task and version. Distinguish unknown or unlabeled outcomes from negative labels. Keep revisions auditable. Acceptance: review survives restart; a correction appears as evidence but leaves current permission unchanged.

F04 - Action ledger and execution [P0]

Persist action intent, approval, idempotency key, outcome and error. States: PROPOSED, APPROVED, EXECUTING, SUCCEEDED, FAILED, CANCELLED and UNKNOWN. A network timeout with uncertain external outcome becomes UNKNOWN and is reconciled before retry. Stage 1 only executes a local sandbox action.

Acceptance: double-clicking Approve or retrying a request produces one sandbox action. Only SUCCEEDED entries count as handled. FAILED and UNKNOWN remain visible. A denied action cannot bypass approval through a direct backend request.

F05 - Monitoring view [P0]

Support task filters, task detail and linked action evidence in Live Analysis. Display totals from the same persistent records used by the queue. Acceptance: imported, reviewed and executed counts reconcile after a restart; stale and failed runs are distinguishable.

F06 - Permission enforcement [P0, Stage 2]

A permission names an action, eligible population, exact configuration, evidence receipt and validity conditions. Missing, stale, mismatched or invalid permission defaults to no AUTO. SUGGEST still requires human approval. QUIET takes no action and may retain evidence for inspection.

Acceptance: changing the model, prompt, retrieval policy or action scope invalidates the prior permission. An engine exception creates a visible failure and cannot fall back to a permissive confidence threshold.

Requirements / decisions and changes

F07 - Experiment definition and execution [P0 for fixed offline tests]

Freeze baseline, candidate, eligible population, outcome rubric, quality target, decision rule, planned checkpoints and exclusions before evaluating. Record execution failures, retries and truncations. Treat complete tasks as the experimental unit unless a justified alternative is declared. Paid execution requires a separately approved spending cap.

For a production experiment, assign one arm per whole task and log assignment probability. Historical imports may inform diagnostics; a causal claim requires appropriate assignment or explicit observational assumptions. Acceptance: the experiment can be reproduced from its manifest and saved results.

F08 - Decision receipts [P0]

Store the verdict and reasons with immutable evidence references, configuration hashes, metric definitions, uncertainty method and test version. Separate statistical eligibility from human approval and activation.

Acceptance: opening a receipt shows exactly which evidence produced it; recalculation creates a new receipt instead of rewriting history.

F09 - Correction gate [P0, Stage 3]

A correction creates a candidate version. Compare it with baseline on held-out tasks, displaying fixed, broken, unchanged and unresolved outcomes. Reserve fresh confirmation data after prompt tuning. KEEP requires the predeclared benefit and regression criteria; DISCARD reflects a violated rule; otherwise return UNPROVEN.

Acceptance: a deliberately harmful candidate is blocked by the defined fixture test; an uncertain result stays unproven. Approval activates a version only after the gate, and rollback restores the previous version and applicable permissions.

F10 - Cost decision [P1]

Compare complete configurations using cost per attempted task and quality. Include input, output, thinking, cache creation/reads, tools and retries when available; disclose omitted cost. Show cost per successful task only alongside the definition and denominator. Distinguish billed cost, provider-derived estimate and simulation.

Acceptance: the same task is not charged once per candidate pair. Zero observed regressions cannot yield a zero-width quality uncertainty claim. Repriced cache projections, when implemented later, remain separate from observed bills.

F11 - Approval and recovery [P0]

Support recommendation approval/decline, permission pause and version rollback. Store actor, time, reason and prior state. The demo may identify one local reviewer; remote shared deployment needs authentication.

Acceptance: approval cannot activate stale evidence and rollback is demonstrable.

F12 - Generic adapters and exports [P1/P2]

Export evidence and receipts in documented JSON/CSV. Later add an HTTP/SDK adapter, dry-run validation and a second real agent workflow. Do not advertise universal plug-and-play integration before testing a second adapter.

What counts as enough evidence

Freeze the question before seeing the answer

The product must distinguish model confidence, measured precision and a valid permission. Confidence is a model output; precision is an observed metric; permission is a scoped decision under an approved evaluation policy. Proposed demo target: 90% precision for one narrowly defined action. The archive also contains a 95% default, so the team must choose one target before evaluation. Neither is a universal safety threshold.

Representative evaluation

The prior 60-ticket hardness set balances classes and inserts known targets when retrieval misses. It is diagnostic evidence only. Permission evaluation must use the actual deployed retrieval process, no injected truth, and a predeclared sample from the eligible stream. Report retrieval failure separately from judgment failure and measure end-to-end outcomes.

Candidates from one ticket are not independent tasks. Sibling issues may also share information. Declare the inference unit and account for clustering or narrow the claim to the sampled cluster population. Use as-of metadata where available; otherwise disclose the limits of a retrospective snapshot.

Missing Jira links do not prove a negative. Blindly review a preselected sample and represent unresolved truth as unknown. LLM graders may assist, but require separate calibration and review; a doubly robust estimator does not automatically correct an inaccurate grader.

Conservative decisions and repeated testing

Choose a reviewed uncertainty method and allocate error across action types, candidates and planned looks. Do not repeatedly reuse a holdout until a result passes. Small samples, unsupported populations, configuration mismatch or invalid labels should produce insufficient evidence or refusal.

The supplied paired-bootstrap comparison and learning-gate intervals require review before production claims, especially zero-disagreement cases and conditional-to-unconditional interval interpretations. The event build may use one locked, simpler validated test instead of reconstructing the old prototype estimator.

Interpretations shown to users

CERTIFY: candidate satisfies the declared quality and cost criteria within scope. **REJECT:** evidence supports a violated criterion. **INSUFFICIENT:** valid comparison but uncertainty remains. **REFUSE:** comparison is not supported by available data or constraints.

Never interpret 'not significantly worse' as proof of acceptable quality. A projected number of additional tasks requires explicit assumptions and sensitivity bounds; otherwise show the missing evidence without a countdown. No AUTO outcome is a valid and potentially compelling result.

Proposed implementation shape

Architecture appropriate to the team

Retain the event-built Python / Streamlit foundation after inventorying it. Separate the adapter, event store, evaluator, permission service and sandbox executor as modules with clear boundaries. Both dashboards read the same records. A modular monolith is sufficient for the event; a service split is not a prerequisite.

Flow: adapter -> task and event records -> reviewer/evaluation -> decision receipt -> approval -> permission or version activation -> executor -> action outcome -> monitoring. Evaluation must not directly invoke a write action. Enforcement rechecks permission at execution time.

Minimum logical entities

Project / Agent / Configuration: identifiers, version hashes, settings, retrieval/action schema versions and created time. Secrets are referenced, never embedded.

Task / Run / Event: task identity, arm assignment if applicable, timestamp, input reference, output/action proposal, status, usage fields and provenance. Preserve attempted runs and errors.

Review: task, rubric version, reviewer, judgment, unknown flag, reason and revision link. **Experiment:** frozen manifest, sample definition, version IDs, checkpoints and lifecycle status.

DecisionReceipt: experiment, verdict, method version, estimates/bounds, exclusions and evidence pointers.

Permission: receipt, eligible scope, exact version, lifecycle and invalidation conditions.

Action: idempotency key, target, requested payload, approval reference, execution state and reconciliation fields.

Change: old/new configuration, decision, approver and rollback pointer.

Persistence and concurrency

Recommended Stage 1: an explicit transactional local store, such as SQLite, for one local process. Preserve supplied data files as read-only inputs. Use database uniqueness for action idempotency, not an in-memory flag. A cloud app's ephemeral filesystem is not accepted durable storage.

For Databricks, first verify available workspace capabilities, authentication and supported durable storage. Use a real supported persistence path and record what is actually connected. Do not assume the app, model endpoints or storage permissions are available merely because setup docs mention them.

General integration contract

An adapter supplies task identity, version, input reference, output/proposed action, outcome/rubric, latency and cost provenance. Assay returns a scoped decision or action policy. Integration also needs allowed actions, identity and retry semantics; an arbitrary chatbot URL does not provide these automatically.

Reliability, privacy and control

N01 - Fail closed and recover [P0]

Evaluation errors, missing permissions, stale versions and unavailable persistence prevent AUTO. Reviews and actions survive an application restart. Failed jobs can resume without duplicating evidence or side effects. A rollback must restore the selected prior configuration; incompatible permissions remain invalid rather than being resurrected blindly.

N02 - Secrets and sensitive data [P0]

Keep keys outside version control in the approved environment or secret store. Do not log them or embed them in exports. The event demo uses public data and sandbox actions. Later customer adapters must minimize retained text, support configurable redaction and document retention/deletion. Do not copy private chatbot conversations into a public demo.

N03 - Access and audit [P0 locally; P1 for shared use]

Record who approved changes. A local single-user demo must be labeled as such. Shared deployment requires authenticated users and backend enforcement of reviewer/operator privileges before exposing write controls. Append audit events for review, approval, activation, pause, execution and rollback.

N04 - Budget and cancellation [P0]

Planning and local recorded-data testing cost zero in external model calls. Any live evaluation needs explicit approval of provider, configuration, maximum calls and money cap. Define reservations, reconciliation and a stop control before unattended paid batches. Unknown call cost must not be treated as zero. No automatic paid fallback.

N05 - Performance targets [proposed, not measured]

On a documented demo machine with a paginated 10,000-task fixture: aim for p95 dashboard navigation under two seconds excluding model execution. Show long-running tests asynchronously with progress and cancellation; never freeze the UI waiting on a model. Report the actual measured configuration and result before claiming compliance.

N06 - Accessibility and legibility [P0]

Use text labels with status colors, readable contrast, keyboard-accessible controls and plain-language explanations. Show why a button is disabled. Avoid presenting technical diagnostics as the user's primary workflow; keep method detail available in evidence views.

N07 - Deployment and backup [P0]

Document local start/stop and a clean-start demo. Verify persistent records after restart and export a backup. Keep a labeled recorded walkthrough if the venue network fails. An unavailable Databricks workspace is an integration blocker to disclose, not a reason to relabel local execution as cloud execution.

How we know the product works

Product acceptance scenarios

A1 / F01-F05: load recorded evidence, inspect a case, approve a sandbox action, reload the app and see the same review and exactly one completed action. Retry and restart must not inflate counts.

A2 / F06-F08: evaluate a locked permission experiment. Inspect a receipt and try a version mismatch, missing labels and an engine failure. None may create AUTO permission. A synthetic passing fixture demonstrates state handling only; it is not real-world validation.

A3 / F09-F11: propose a correction, compare versions on untouched tasks, inspect fixes and regressions, approve an eligible candidate, then roll back. Test a deliberately harmful candidate and an uncertain candidate as separate cases.

A4 / F10: aggregate per-task usage including retries; reconcile against source records. Label API-equivalent estimates separately from invoices. Assert that a zero-disagreement small sample remains uncertain.

A5 / F12: after the event, connect a second supported workflow through the common contract without changing the decision engine's domain logic.

Proposed launch metrics

Traceability: 100% of displayed executed actions link to a persisted result and approval or valid permission.

Idempotency: zero duplicate side effects in retry/restart fixtures. **Evidence integrity:** no injected targets in the permission set; all evaluation records have version and provenance.

Decision validity: uncertainty and scope appear on every decision receipt; invalid evidence never enables AUTO. Real qualifying permission rate is measured, not targeted upward at the expense of rigor.

Usability: at least one teammate unfamiliar with implementation completes the review-to-action path from the instructions. Record completion time and confusion; do not invent a user-study success rate.

Judge-facing demonstration

Begin with a real recorded case. Inspect the proposed action, approve it in the sandbox and reveal the ledger entry. Then show a frozen decision receipt, a candidate correction and a blocked regression. A judge may enter a fresh ticket only if an authorized live backend is available; otherwise state that fresh inference is unavailable and use labeled fixtures.

Art of the Break

Record a failure caused by the team's own agent design: overconfident part-of classification, a harmful correction, or configuration-induced excess thinking cost. Keep the failed case, expected behavior, reproduced outcome, detection rule, fix and independent retest. Select only a failure actually reproduced; a proposed stress case is not an observed result.

Scope and open decisions

Event release versus full product

Event must-have: persistent review and sandbox actions; two coherent dashboard views; honest evaluation data; scoped permissions or an honest insufficient-evidence result; correction gate; versioned receipts; working demo and submission evidence.

Conditional event additions: configuration-specific cost comparison and a minimal real Databricks integration, depending on evidence, access and remaining time. Databricks challenge eligibility requires confirming the actual integration meets the organizer's expectations.

Later: generic agent onboarding, production connectors, prospective experiments, validated cache simulation, holdbacks and monitoring, authenticated multi-user service, billing and operational hardening. Do not implement all of these before the event.

Non-goals: rebuilding Databricks, certifying arbitrary agents from their URL, claiming global cost optimality, automatically retraining from every correction, or silently operating on external customer systems.

Decision register before dependent work

D1 / team: approve Stage 1 and this proposed scope. **D2 / evidence owner:** select the quality target, action family, sampling unit and uncertainty policy before collecting permission evidence. **D3 / operator:** verify Databricks access, model entitlement and durable storage. **D4 / owner:** approve any live-call budget separately. **D5 / team:** select the exact demo failure and submission cutoff. **D6 / owner:** define production risk tolerances before any external writes.

Default while unresolved: local recorded mode; no external model spending; sandbox-only writes; no unvalidated AUTO; no unsupported statistical claim. Resolve routine implementation details during the approved stage without repeatedly requesting permission.

Sources and current-state confidence

Primary source: assay-handover-1600.zip, reviewed 26 September 2026. Key references: START_HERE_CLAUDE.md; docs/thinking/03-brief-v3-fact-check.md; docs/SCHEMA.md; docs/DATABRICKS_SETUP.md; app/app.py and app_data.py; assay_engine/bands.py, gate.py and compare.py; scripts/judge_eval.py; the hardness-60 summary.

The archive contains an event-built app and reports 40 tests. Those tests were not independently rerun during planning. It contains 37,853 tickets and 656 candidate-pair judgments from 60 tickets across two models; those judgments are not 656 independent tasks. Cost findings in that diagnostic set are configuration-specific.

This PRD proposes requirements, not a claim that they are already implemented. It uses the teammate handover as the foundation; prior device-only prototype work is not the implementation baseline. Vendor availability and challenge requirements still need current workspace/organizer verification. Document approval does not itself authorize paid calls, external writes, commits or pushes.